

Programmation Orientée Objet

Travaux Dirigés ## Licence 2 - S4 - UA

**** ⚠ Règle importante : IA autorisée avec responsabilité ****

- Utilisation d'IA AUTORISÉE et ENCOURAGÉE pour apprendre
- OBLIGATION d'expliquer et comprendre chaque ligne
- Interrogations orales aléatoires sur votre code
- Documenter quand et comment l'IA a été utilisée

MODULE 1 - FONDAMENTAUX POO

TD1 - Introduction à la POO et premières classes (2h)

🎯 Objectifs

- Comprendre le paradigme objet vs procédural
- Créer des classes simples
- Manipuler attributs et méthodes

Exercice 1.1 : Du procédural à l'objet (30min)

****Contexte**** : Gestion d'un compte bancaire en procédural puis en POO.

****Code procédural existant : ****

```
` `` python
```

```
# Version procédurale (à NE PAS faire)
```

```
compte_numero = "FR7630001007941234567890185"
```

```
compte_titulaire = "Alice Dupont"
```

```
compte_solde = 1000.0
```

```
compte_historique = []
```

```
def deposter(montant):
```

```
    global compte_solde, compte_historique
```

```

compte_solde += montant
compte_historique.append(f"Dépôt: +{montant}€")
return compte_solde

```

```

def retirer(montant):
    global compte_solde, compte_historique
    if montant > compte_solde:
        return False
    compte_solde -= montant
    compte_historique.append(f"Retrait: -{montant}€")
    return True
` ``

```

****À FAIRE : ****

1. Identifiez 5 problèmes avec cette approche procédurale
2. Créez une classe `CompteBancaire` avec :
 - Attributs : `numero`, `titulaire`, `solde`, `historique`
 - Méthodes : `deposer(montant)`, `retirer(montant)`, `afficher\_solde()`, `afficher\_historique()`
3. Implémentez une validation : solde ne peut pas être négatif
4. Ajoutez une méthode `virement(montant, autre\_compte)` pour transférer de l'argent

****Questions de réflexion : ****

- Pourquoi est-ce plus facile de gérer plusieurs comptes en POO ?
- Comment la classe protège-t-elle les données ?
- Que se passe-t-il avec les variables globales si on a 100 comptes ?

Exercice 1.2 : Classes et instances (40min)

****Créez une classe `Etudiant` complète****

```

` `` python
class Etudiant:
    # Attribut de classe (partagé)
    universite = "Université des Antilles"
    nombre_etudiants = 0

    def __init__(self, nom, prenom, numero_etudiant, filiere):
        # TODO: Implémenter
        pass

    def ajouter_note(self, matiere, note):
        # TODO: Valider note entre 0 et 20
        pass

    def calculer_moyenne(self):

```

```

        # TODO: Moyenne de toutes les notes
        pass

def calculer_moyenne_matiere(self, matiere):
    # TODO: Moyenne d'une matière spécifique
    pass

def est_admis(self, seuil=10):
    # TODO: Vérifier si moyenne >= seuil
    pass

def obtenir_mention(self):
    """Retourne la mention selon la moyenne"""
    # Passable: 10-12, Assez bien: 12-14, Bien: 14-16, Très bien: 16+
    pass

def __str__(self):
    # TODO: Affichage lisible
    pass
` ``

**Tests à réaliser :**
` `` python
# Test création
alice = Etudiant("Dupont", "Alice", "E12345", "Informatique")
bob = Etudiant("Martin", "Bob", "E12346", "Informatique")

# Test attribut de classe
print(Etudiant.nombre_etudiants) # 2
print(alice.universite) # "Université des Antilles"

# Test notes
alice.ajouter_note("POO", 15)
alice.ajouter_note("Web", 14)
alice.ajouter_note("POO", 16) # 2ème note en POO
alice.ajouter_note("Mobile", 13)

print(alice.calculer_moyenne()) # Moyenne générale
print(alice.calculer_moyenne_matiere("POO")) # 15.5
print(alice.est_admis()) # True
print(alice.obtenir_mention()) # "Bien"
` ``

**Challenge :**
- Ajoutez une méthode `comparer_avec(autre_etudiant)` qui compare les moyennes
- Créez une classe `Promotion` qui gère une liste d'étudiants

```

- Implémentez des statistiques de promotion (moyenne, taux de réussite, meilleur étudiant)

Exercice 1.3 : Méthodes spéciales Python (50min)

****Contexte**** : Créer une classe `Vecteur2D` mathématiquement correcte.

```
` `` python
class Vecteur2D:
    def __init__(self, x, y):
        self.x = x
        self.y = y

    def __str__(self):
        return f"Vecteur({self.x}, {self.y})"

    def __repr__(self):
        return f"Vecteur2D({self.x}, {self.y})"

    def __add__(self, autre):
        """Surcharge de l'opérateur +"""
        # TODO: Retourner un nouveau vecteur (x1+x2, y1+y2)
        pass

    def __sub__(self, autre):
        """Surcharge de l'opérateur -"""
        pass

    def __mul__(self, scalaire):
        """Multiplication par un scalaire"""
        #  $v * 3 = \text{Vecteur}(x*3, y*3)$ 
        pass

    def __rmul__(self, scalaire):
        """Multiplication à droite :  $3 * v$ """
        return self.__mul__(scalaire)

    def __eq__(self, autre):
        """Égalité de vecteurs"""
        pass

    def __abs__(self):
        """Norme du vecteur :  $|v| = \sqrt{x^2 + y^2}$ """
        import math
        pass
```

```

def __neg__(self):
    """Opposé du vecteur : -v = Vecteur(-x, -y)"""
    pass

def produit_scalaire(self, autre):
    """Produit scalaire :  $v_1 \cdot v_2 = x_1 x_2 + y_1 y_2$ """
    pass

def angle_avec(self, autre):
    """Angle entre deux vecteurs (en degrés)"""
    import math
    #  $\cos(\theta) = (v_1 \cdot v_2) / (|v_1| * |v_2|)$ 
    pass

```

****Tests mathématiques : ****

```

` `` python
v1 = Vecteur2D(3, 4)
v2 = Vecteur2D(1, 2)

```

Opérations

```

v3 = v1 + v2 # Vecteur(4, 6)
v4 = v1 - v2 # Vecteur(2, 2)
v5 = v1 * 2 # Vecteur(6, 8)
v6 = 3 * v1 # Vecteur(9, 12)

```

Comparaisons

```

print(v1 == Vecteur2D(3, 4)) # True
print(abs(v1)) # 5.0 (norme)
print(-v1) # Vecteur(-3, -4)

```

Opérations mathématiques

```

print(v1.produit_scalaire(v2)) # 11
print(v1.angle_avec(v2)) # Angle en degrés
` ``

```

****Challenge IA : ****

1. Demandez à l'IA de générer cette classe
2. Testez TOUS les cas limites :
 - Division par zéro dans angle_avec
 - Vecteurs nuls
 - Multiplication par 0
3. Corrigez les bugs et améliorez